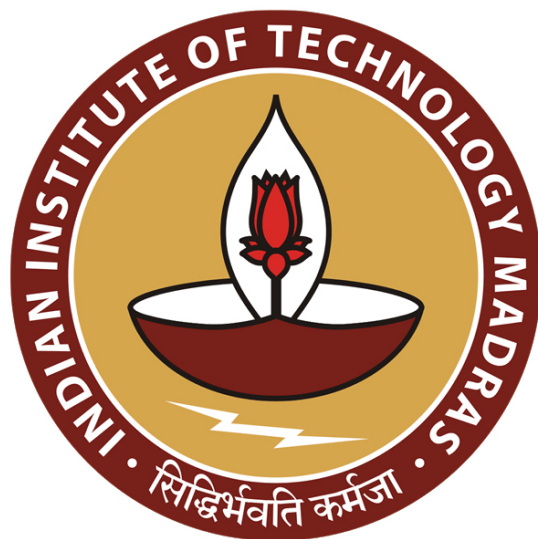




IIT Madras
ONLINE DEGREE

Database Management Systems
Professor Partha Pritam Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Relational Database Design/8: Case Study

Welcome to Module 28 of Database Management Systems course in the IIT Madras online B.Sc. program.

(Refer Slide Time: 00:31)

Module Recap

- Learnt how to decompose a schema into 3NF while preserving dependency and lossless join
- Learnt how to decompose a schema into BCNF with lossless join

Database Management Systems

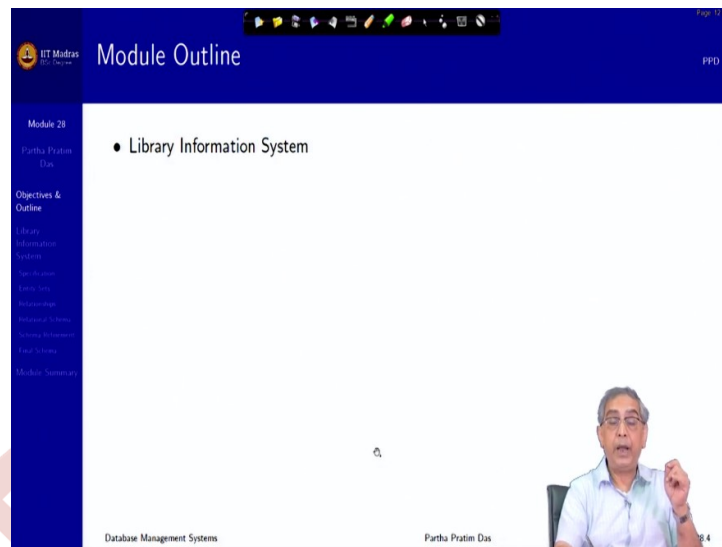
We have been discussing about designing of relational database schemas and we have made a lot of studies into that and using the theory of functional dependency we have learned how to model and design good schema with decomposition into 3NF or BCNF.

(Refer Slide Time: 00:58)

Module Objectives

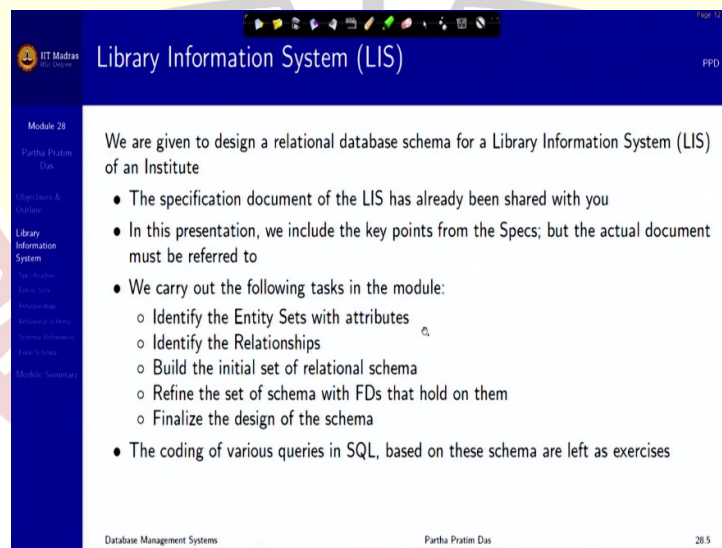
- To design the schema for a Library Information System

Database Management Systems



So, it is time to just kind of take a step back and start, we have done a lot of theory. So, what I want to do in this module is take a step back stop discussing theory, but apply what we have learned. So, you must have received the specification statement for a library information system. It is a simple English language document which describe, how a library information system is required.

(Refer Slide Time: 01:35)



Now, when you go through this presentation, please keep that document available with you, alongside with you. I will accept the key points from the document here. But it is also important to understand how to accept the key points. There will be a tutorial on this part as well as to from the specification, how do you come to a kind of more manageable and you know concrete description of the specification.

So, first before we proceed let us identify the tasks. So, we are given to design a relational database schema for a library information system of an institute. The specification system document you have. And in this presentation, we include the key points from the specification as expert as we as I said, and the tasks are naturally from this description, we have to first identify the entity sets and their attributes.

We have to identify the relationships that is a that is basically we have to carry out the Entity Relationship Modelling for the specification. Once we have the Entity Relationship Model, we will transform that into a set, initial set of relational schema. So, we will generate a number of relational schema that come from the ER model, then we will have to refine the set of schema with the functional dependencies that hold on them.

So, that our target would be to okay, achieve BCNF in every case, if we cannot, we will try to do 3NF. But considering the functional dependencies, which also the functional dependencies will not be given in the document, the functional dependencies will be hidden in the statements of the document and we will have to extract those understanding and put them in the notion, notation of the functional dependencies.

And using those we will have to keep decomposing every relational scheme to make sure they are in BCNF. And if not, at least in 3NF. And beyond that also we will have to do something is we will alongside check on what are the possible coding of some of the typical queries, all queries obviously, the designer will not code that's a application programmer's job.

And we might see that my design may need some change. So that certain query can actually be efficiently written. This is a factor which we have not talked about earlier, we have only been talking about the theory part of it, but there are pragmatics part of it. The pragmatics say that well, you may have a very good design, but unless you can efficiently write queries, which is the target, the design will not be appreciable.

So, that is the first final stage that we will make some refinements like this and finalize the design. Of course, then the remaining part of the task is to actually code the various queries which we will not be doing in the module, which I will leave, leave for you as exercise to practice and we have already done a whole lot of SQL query programming.

(Refer Slide Time: 05:06)

LIS Specs Excerpts

- An institute library has 200000+ books and 10000+ members
- Books are regularly issued by members on loan and returned after a period.
- The library needs an LIS to manage the books, the members and the issue-return process
- Every **book** has
 - title
 - author (in case of multiple authors, only the first author is maintained)
 - publisher
 - year of publication
 - ISBN number (which is unique for the publication), and
 - accession number (which is the unique number of the copy of the book in the library)
 - *There may be multiple copies of the same book in the library*

Database Management Systems Partha Pratim Das 28.6

So, let me run through the specification quickly before I get into the design. So, it says that the institute library has some these numbers are not important, it just says that there are large number of books and large number of members and books are naturally regularly issued by members on roll and returned after a period. So, you immediately get an information that there will have to be a issue relationship, issue process and that associated with an issue there is a time period.

So, there is a date of return kind of thing that will be involved. And the library needs an LIS. Now if you go through the sentences, you will see that it says that every book has a I mean, it was written in the long longhand English what I have just I have just kind of bulleted them so that it starts looking more like the ER information. So, a book very clearly, the book has to be an entity has a title, author, in case of multiple author the first author so it says that it is not multivalued.

It has publisher, year of publication, ISBN number which is unique for a publication and accession number. The difference between them is a say if you consider Silberschatz, Korths book, it has one ISBN number so, all copies of that book has the same ISBN number that is a publication number.

And accession number is if there are 5 copies of that book in the library, then naturally those 5 copies have to be distinguished. So, they will be given separate separate number. So, that is called the accession number. And that is what is highlighted at the end, there may be multiple copies of the book.

(Refer Slide Time: 06:52)

The slide is titled "LIS Specs Excerpts (2)" and is part of a presentation from IIT Madras. It lists the following specifications:

- There are four categories of **members** of the library:
 - undergraduate students
 - post graduate students
 - research scholars, and
 - faculty members
- Every **student** has
 - name
 - roll number
 - department
 - gender
 - mobile number
 - date of birth, and
 - degree
 - undergrad
 - grad
 - doctoral

A blue bracket groups the attributes from "name" to "date of birth, and". Another blue bracket groups the "degree" options: "undergrad", "grad", and "doctoral".

Then you have members, four categories of that undergraduate, postgraduate students, research students and faculty members. Student has this is also written in terms of a description I have extracted out and put it in terms of bullets, which looks somewhat like somewhat like your ER entity relationship entity attribute kind of. So, this is going to be an entity and these are going to be the attributes and these three are naturally value constraints.

So, you are saying degree can be undergrad, grad or doctoral nothing else. So, this is a value constraint, we have talked about value constraints earlier in terms of semester and all that examples. So, similar things you have here. So, this is a.

(Refer Slide Time: 07:41)

The slide is titled "LIS Specs Excerpts (3)" and is part of a presentation from IIT Madras. It lists the following specifications:

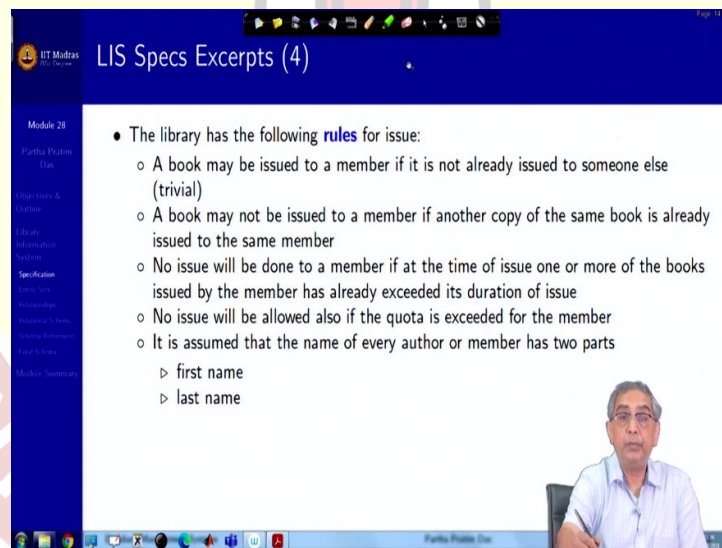
- Every **faculty** has
 - name
 - employee id
 - department
 - gender
 - mobile number, and
 - date of joining
- Library also issues a unique **membership number to every member**. Every member has a **maximum quota** for the number of books she / he can issue for the maximum duration allowed to her / him. Currently these are set as:
 - Each undergraduate student can issue up to 2 books for 1 month duration
 - Each postgraduate student can issue up to 4 books for 1 month duration
 - Each research scholar can issue up to 6 books for 3 months duration
 - Each faculty member can issue up to 10 books for six months duration

Blue brackets group the four membership rules, highlighting the "books" and "duration" for each category.

Then, then you have faculty with all these properties that faculty has again described, then it makes a very interesting information in terms of the membership, it says library also issues a unique membership to every member. So, you get to know that there has to be a kind of a member entity which has a membership number and every member has a maximum quota for the number of books at a time how many can you issue and here in when you look here, here in it says what is the quota for each and every type of member.

So, it talked about the type of membership earlier. So, the quota is not based on individual member but it is based on the type of member. These are not explicitly, explicitly tabulatively stated, but these are what you can read, make out from the. And you can know that the quota basically talks about two things one is the number of books and other is a period of, period for which you can actually retain the book. So, so, this is basically you are seeing the entire document.

(Refer Slide Time: 09:03)



The screenshot shows a presentation slide titled "LIS Specs Excerpts (4)". The slide content is as follows:

- The library has the following **rules** for issue:
 - A book may be issued to a member if it is not already issued to someone else (trivial)
 - A book may not be issued to a member if another copy of the same book is already issued to the same member
 - No issue will be done to a member if at the time of issue one or more of the books issued by the member has already exceeded its duration of issue
 - No issue will be allowed also if the quota is exceeded for the member
 - It is assumed that the name of every author or member has two parts
 - ▷ first name
 - ▷ last name

The slide is part of a presentation titled "Module 28: Partha Prasad Das" and is displayed in a window titled "LIS Specs Excerpts (4)". A small video inset in the bottom right corner shows a man speaking.

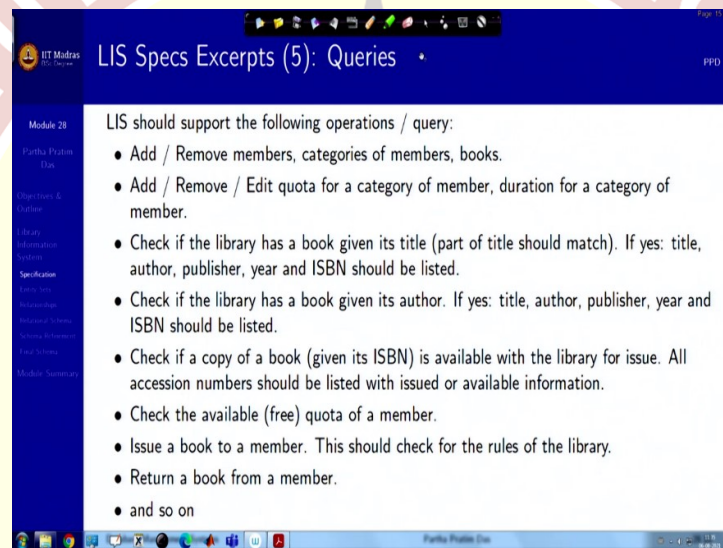
I am just reading out in forms of excerpts in a compact form. And finally, it says that there are some ground rules. A book maybe issued to a member if it is not already issued to someone else, which is trivial. A book may not be issued to a member if another copy of the book is already issued to the member. So, this is a constraint, this is a constraint being specified.

No issue will be done to a member if at the time of issue, one or more books issued by the member has already exceeded its duration. So, you will have to decide whether, whether you

would like to keep this in the database constraints with how convenient is that or whether these are constraints for the application program. But these are constraints which they are talking about.

No issue will be allowed also if quota is exceeded for the member, this should be. So, this kind of a constraint will also have to be in place and it is assumed that every author or member has two parts in their name, first name and last name. So that this will help us get the atomic domain. Because it says that this is name is a composite attribute we know and it says what are the components we will flatten out and make it simply atomic.

(Refer Slide Time: 10:19)



Now, it also talks about certain representative queries like adding, removing members, categories of members, books and so on. Adding, removing, editing quota of category of members, duration and so on. So, all these are basically you have to take each one of them and you will have to subsequently, I mean you will have to see whether your design can easily, conveniently cater to writing the sequel query for this.

If not, then you have to revise the design and if it does, then at the end, you will have to sit down and write all these queries check if a library has a, so, there are quite a number of them, will take just representative look at some of these.

(Refer Slide Time: 11:07)

LIS Entity Sets: books

- Every book has title, author (in case of multiple authors, only the first author is maintained), publisher, year of publication, ISBN number (which is unique for the publication), and accession number (which is the unique number of the copy of the book in the library). There may be multiple copies of the same book in the library
- Entity Set:
 - **books**
- Attributes:
 - title
 - author_name (composite)
 - publisher
 - year
 - ISBN_no
 - accession_no

So, this is about the specification. So, you have got the specifications. Now, you are doing the design in the ER model, I am not presenting the design as in the diagram, as in terms of the diagram rather I am presenting in terms of a text representation. So, very clearly these are the books and these are the different, this is the entity set and these are the different attributes highlighted that this is composite.

So, this will have to be broken up to get this relation to 1NF and what is given above is the relevant sentences from the specification which you are translating into this kind of a ER representation.

(Refer Slide Time: 11:53)

LIS Entity Sets (2): students

- Every student has name, roll number, department, gender, mobile number, birth, and degree (undergrad, grad, doctoral)
- Entity Set:
 - **students**
- Attributes:
 - member_no – is unique
 - name (composite)
 - roll_no – is unique
 - department
 - gender
 - mobile_no – may be null

You have students which has all these different attributes, this entity set having all these attributes and you have a value constraint on this coming from here, which we will have to remember to take care in the later part of the design.

(Refer Slide Time: 12:14)

LIS Entity Sets (3): faculty

- Every faculty has name, employee id, department, gender, mobile number, and date of joining
- Entity Set:
 - faculty
- Attributes:
 - member_no – is unique
 - name (composite)
 - id – is unique
 - department
 - gender
 - mobile_no – may be null
 - doj

LIS Entity Sets: books

- Every book has title, author (in case of multiple authors, only the first author is maintained), publisher, year of publication, ISBN number (which is unique for the publication), and accession number (which is the unique number of the copy of the book in the library). There may be multiple copies of the same book in the library
- Entity Set:
 - books
- Attributes:
 - title
 - author_name (composite)
 - publisher
 - year
 - ISBN_no
 - accession_no (unique)

IIT Madras

LIS Entity Sets (2): students

Module 26

- Every student has name, roll number, department, gender, mobile number, and degree (undergrad, grad, doctoral)
- Entity Set:
 - students
- Attributes:
 - member_no – is unique
 - name (composite)
 - roll_no – is unique
 - department
 - gender
 - mobile_no – may be null

Very simple as you see the faculty there is nothing specific to mention except that again name is a composite one. In fact, you can in earlier ones also another point that you should have noted for example, here you can note that this is unique. ER model allows you to note that, because certainly you can have multiple copies, multiple books with the same title, multiple books with the same author, naturally, you can have multiple copies of Korth.

So, which means that all of these can be repeated in the database except the stamping of the specific copy which is accession number, so, accession number is unique. So, this identification of uniqueness is very important as you understand because you have to after having done the basic extraction of entity and attributes, you will also have to mark the key attributes in the process.

So, similarly in in in students, this is member number it is roll number. So, basically both of them a student will have a unique roll number. So that is what you are noting, the naturally being a member of this library, you will have a unique member number. So, that is also another key. So, it has two keys. Similarly, faculty will have member number as unique and ID as unique, another constraint written here mobile number maybe null.

And here we are assuming that things like date of joining, date of birth is of date type, since you have a date type, so that can be taken as an atomic component. You do not need to consider that to be composite.

(Refer Slide Time: 14:08)

LIS Entity Sets (4): members

- Library also issues a unique membership number to every member. There are four categories of members of the library: undergraduate students, post graduate students, research scholars, and faculty members
- Entity Set:
 - members
- Attributes:
 - member_no
 - member_type (takes a value in ug, pg, rs or fc)

Then members, where you have member number, which certainly is unique. And then you have a member type, which has a constraint on the values it can take UG, PG, research scholar or faculty.

(Refer Slide Time: 14:23)

LIS Entity Sets (5): quota

- Every member has a maximum quota for the number of books she / he can issue for the maximum duration allowed to her / him. Currently these are set as:
 - Each undergraduate student can issue up to 2 books for 1 month duration
 - Each postgraduate student can issue up to 4 books for 1 month duration
 - Each research scholar can issue up to 6 books for 3 months duration
 - Each faculty member can issue up to 10 books for six months duration
- Entity Set:
 - quota
- Attributes:
 - member_type
 - max_books
 - max_duration

unique (handwritten note pointing to member_type)

Now you have a quota. So, this is what I was explaining earlier that from this statement you are I mean, it is not clearly quota, quota is not a, I should say it is not a tangible entity, like faculty, student or book are tangible entities you can touch them. But quota is an intangible entity. It is a notion it is a kind of permission that you give. So, it is embedded in this description.

So, you identify this is a quota and what does it, what all does it depend on from the statement, a quota, to describe a quota you need to know the member type, you need to know how many books and you need to know for how much time. So, these turn out to be your three attributes which are not clear, like you had in the description of student or book or faculty that is how you get these.

Now, the question obviously is what is the unique? Obviously, member type will have to be unique because you cannot have two different quota for the same member type. So, this is your quota design.

(Refer Slide Time: 15:33)

The screenshot shows a presentation slide titled "LIS Entity Sets (6): staff". The slide content includes:

- *Though not explicitly stated, library would have staffs to manage the LIS*
- Entity Set:
 - staff
- Attributes: (speculated – to ratify from customer)
 - name (composite) ✓
 - id – is unique ✓
 - gender ✓
 - mobile_no – *maybe null*
 - doj

Handwritten notes in blue ink are present: "maybe null" is written next to "mobile_no", and checkmarks are placed next to "name (composite)", "id – is unique", and "gender". A small video inset in the bottom right corner shows a man speaking.

Then, it is not mentioned, but you will I mean this these are, these kinds of gaps are very typical in a specification. That when you get the specification, you start doing the design you will ask, okay, all are there students are there, undergraduate, postgraduate, research scholar, faculty, everybody is there. But how about the staff? How will the library run? There will have to be staff to run the library.

And it is naturally, ridiculous that if the staff who run the library cannot access the library. So, this is kind of possibly we will go back to the client and say that and client say oh my God, I forgot. Thanks for the advice. So, you are trying to not only capture what is written, but at times you are trying to read between the lines in the thought process of the requirement to say that well there has to be staff running.

So, there has to be a staff entity as well. And rest of it is somewhat guesswork, which is speculation, which will have to get ratified from the customer that certainly staff will have a name which is composite in the way, the ID will be there, gender will be there whether the staff is allowed to operate without a mobile or must be with a mobile is a question you will have to ask the customer.

If he or she is allowed to do that, then it can be made to be null, then null will be allowed, date of joining there may be more like you may want to have certain staff positions to decide what kind of membership they will have and so on. So, this is something so, this is something very interesting this is going from specification to the ER model. And what you see is some initially what we said was crisply written in the specification.

Then we saw something like quota which was written in a descriptive form and you have to dig out and then we are talking about something which is not written at all, but you are reading between the lines and putting your thought process and that is the contribution you make as a database designer. Otherwise, if this is missed out later on, you will have to come back and make changes very expensive.

(Refer Slide Time: 17:59)

LIS Relationships

- *Books are regularly issued by members on loan and returned after a period. The library needs an LIS to manage the books, the members and the issue-return process*
- Relationship
 - book_issue ✓
- Involved Entity Sets
 - students / faculty / members ✓
 - ▷ member_no
 - books ✓
 - ▷ accession_no ✓
- Relationship Attribute
 - doi - date of issue
- Type of relationship
 - Many-to-one from books

Naturally, relationships, there could be several types of relationships you can think of, but you are looking at the library aspect like you can think of relationship between author and title of the book who has written what, author and publisher. But those kinds of relationships

you will not, we are not extracting or speculating, but we focus on the fact that it says the books are regularly issued.

So, there will have to be a book issue that is a core purpose of the library and who can issue? A student, faculty or members in general. So, a member number will have to be there for this and what can be issued? Our books and what is the uniqueness on that site is accession number. So, that you know when we create a binary relationship, we have the key on both sides to connect the relationship.

Now, we need a relationship attribute here. Why do we need relationship attribute? Because when the issue is done at that point of time I need to record the date so that later on I can enforce the information on quota. So, this is neither in members relation, DOI is not in members relation nor in the books relation, but it has to come as an relationship attribute we talked about this earlier. And identify the type of relationship, type of relation is naturally many to one because one a book can be issued to only one member, but one member can issue several books.

(Refer Slide Time: 19:51)

The screenshot shows a presentation slide titled "LIS Relational Schema" from IIT Madras. The slide lists the following tables and their attributes:

- books(title, author_fname, author_lname, publisher, year, ISBN_no, accession_no)
- book_issue(members, accession_no, doi)
- members(member_no, member_type)
- quota(member_type, max_books, max_duration)
- students(member_no, student_fname, student_lname, roll_no, department, gender, mobile_no, dob, degree)
- faculty(member_no, faculty_fname, faculty_lname, id, department, gender, mobile_no, doj)
- staff(staff_fname, staff_lname, id, gender, mobile_no, doj)

Handwritten blue annotations are present: checkmarks next to each table name, underlines for primary keys (accession_no, member_no, roll_no, id, staff_fname, staff_lname, id), and a note "m. no" with an arrow pointing to the member_no attribute in the staff table.

So, based on this now you can quickly so this is so up to that point was entity relationship diagram. If you feel more comfortable, you can write them down, I mean, you can draw them down in terms of the ER diagrammatic model, but I have just used the textual version. But from this now, you can very quickly dump this entire information in terms of the relational schema.

So, you have the relational schema books, members, quota, students, faculty coming directly from that this is your speculated one. And this is your, this is coming from your relationship. And in each, you can actually mark out what is the key, this is the key book issue. Are both, do both need to be key? No. You can only have the accession number because there is a many to one relationship, you may not, do not need this.

Member number is a key, member type is a key. Here member number and roll number are keys, two different not together. Faculty number, ID are keys again, separately. Here staff name, I am sorry, not staff name, the staff ID is a key. If we put member number, then that will also be a key. So, this is a very basic first cut relational schema for the entire LIS system.

(Refer Slide Time: 21:30)

LIS Schema Refinement: books

- **books**(title, author_fname, author_lname, publisher, year, ISBN_no, accession_no)
 - ISBN_no → title, author_fname, author_lname, publisher, year
 - accession_no → ISBN_no
 - Key: accession_no
- Redundancy of book information across copies
- Good to normalize:
 - **book_catalogue**(title, author_fname, author_lname, publisher, year, ISBN_no)
 - ▷ ISBN_no → title, author_fname, author_lname, publisher, year
 - ▷ Key: ISBN_no
 - **book_copies**(ISBN_no, accession_no)
 - ▷ accession_no → ISBN_no
 - ▷ Key: accession_no
- Both in BCNF. Decomposition is lossless join and dependency preserving

LIS Schema Refinement: books

- **books**(title, author_fname, author_lname, publisher, year, ISBN_no, accession_no)
 - ISBN_no → title, author_fname, author_lname, publisher, year ✓
 - accession_no → ISBN_no ✓
 - Key: accession_no
- Redundancy of book information across copies
- Good to normalize:
 - **book_catalogue**(title, author_fname, author_lname, publisher, year, ISBN_no) ✓
 - ▷ ISBN_no → title, author_fname, author_lname, publisher, year
 - ▷ Key: ISBN_no
 - **book_copies**(ISBN_no, accession_no) ✓
 - ▷ accession_no → ISBN_no
 - ▷ Key: accession_no
- Both in BCNF, Decomposition is lossless join and dependency preserving

So, what we get into is now get into the refinements. So now we will have to see that well, these are the relations I have got, what are the possible functional dependencies that are implicit in the statements. So, there was a statement saying that, so, we are looking at books, and there was a statement saying that every publication has a unique ISBN number. So, we get the first dependency.

So, this is the first functional first FD that we get, which says that given ISBN number I know title, author, name, author, last name, publisher, year all that and we also have that every book has a unique stamping of accession number. So, this is the other functional dependency. Clearly in terms of the functional dependency, the key now can be formally be shown to be accession number.

Now, the problem that you will get is a Korth, Silberschatz book there are five copies, they have different accession number, but all of these will be common for all these five copies. So, there is redundancy across this book copies. So, what we should do, we should try to normalize. And as we normalize, this is a very straightforward case, which actually by, even by observation, you will be able to do, it is because it is very disjointed part of the information.

So, if you look into this, then this does not violate BCNF, but this violates BCNF. This does not violate BCNF because accession number is a key, but the other one will violate BCNF because ISBN number is not the key. So, if we do a BCNF decomposition, we will get these two relations very clearly. So, both of them now are in BCNF because the two functional dependencies one hold on this, the other holds on this, these are the resultant keys both are in BCNF you can check both the functional dependencies on the decomposition. So, it is a BCNF decomposition lossless and dependency preserving, very happy situation. So, your books relationship schema has been normalized and finalized.

(Refer Slide Time: 24:01)

IIT Madras

LIS Schema Refinement (2): book_issue

Module 28
Partha Pratim Das

- **book_issue**(member_no, accession_no, doi)
 - member_no, accession_no → doi
 - Key: members, accession_no
- In BCNF

Database Management Systems Partha Pratim Das

Then you have book issue which has member number, accession number will decide obviously the date of issue, because why, why that is required, why member number is required, I was saying that accession number itself will do, it is not, it may not because the book, same book may be issued to different members at different times.

So, we will have to look at those aspects but actually it is safe to think that this pair, member number and accession number will decide the date of issue so that is the key, the only functional dependency so this is in BCNF.

(Refer Slide Time: 24:42)

IIT Madras

LIS Schema Refinement (3): quota

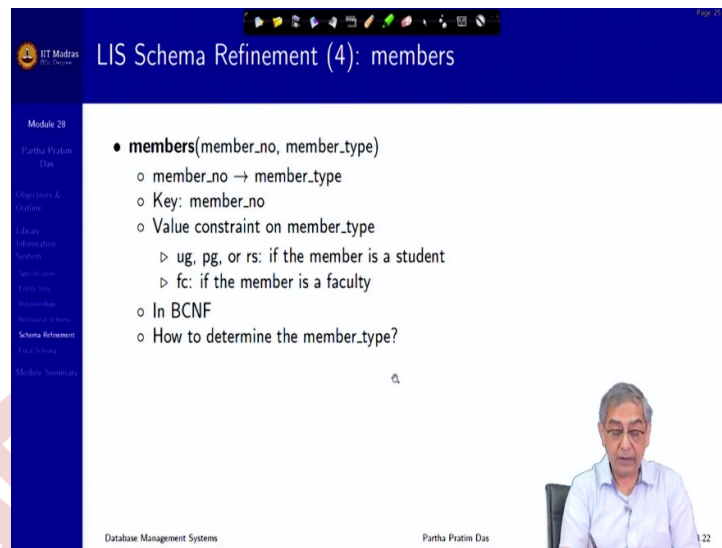
Module 28
Partha Pratim Das

- **quota**(member_type, max_books, max_duration)
 - member_type → max_books, max_duration
 - Key: member_type
- In BCNF

Database Management Systems Partha Pratim Das

Quota, member type will decide maximum books, maximum duration, so member type is a key it is in BCNF.

(Refer Slide Time: 24:53)



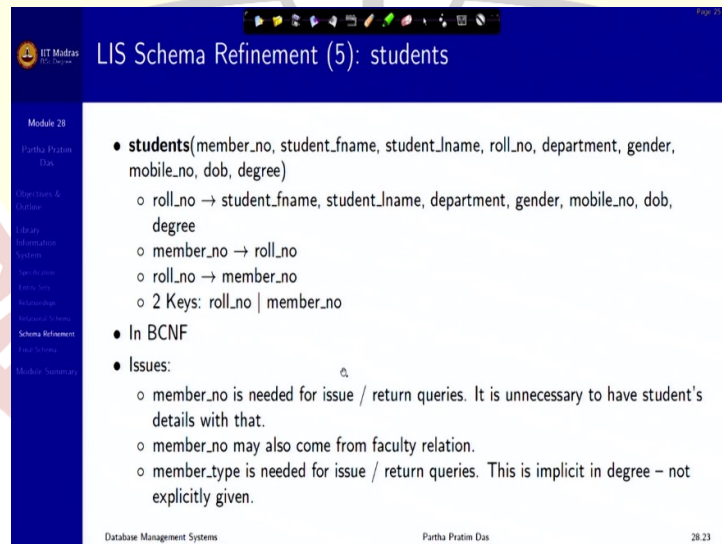
LIS Schema Refinement (4): members

- **members**(member_no, member_type)
 - member_no → member_type
 - Key: member_no
 - Value constraint on member_type
 - ▷ ug, pg, or rs: if the member is a student
 - ▷ fc: if the member is a faculty
 - In BCNF
 - How to determine the member_type?

Database Management Systems | Partha Pratim Das | 22

Members, very easy to see that member number is the key which decides member type and there is value constraint on the member type. But otherwise, this is in BCNF. Certainly, there is a question as to how do you get the member type? So, that question you will have to answer.

(Refer Slide Time: 25:14)



LIS Schema Refinement (5): students

- **students**(member_no, student_fname, student_lname, roll_no, department, gender, mobile_no, dob, degree)
 - roll_no → student_fname, student_lname, department, gender, mobile_no, dob, degree
 - member_no → roll_no
 - roll_no → member_no
 - 2 Keys: roll_no | member_no
- In BCNF
- Issues:
 - member_no is needed for issue / return queries. It is unnecessary to have student's details with that.
 - member_no may also come from faculty relation.
 - member_type is needed for issue / return queries. This is implicit in degree – not explicitly given.

Database Management Systems | Partha Pratim Das | 28.23

Students, roll number decides everything member number decides roll, roll number decides member number. So, there are two keys and everything is in BCNF. The issue is the member number is needed for issue written queries. So, it is unnecessary to have those details with that. So, it is in BCNF, but it is not a good, it may not be a good design, because do you really need all this information here it is in BCNF because it uniqueness is there. But do you really need to keep all that.

(Refer Slide Time: 25:55)

LIS Schema Refinement (6): faculty

- **faculty**(member_no, faculty_fname, faculty_lname, id, department, gender, mobile_no, doj)
 - $id \rightarrow \text{faculty_fname, faculty_lname, department, gender, mobile_no, doj}$
 - $id \rightarrow \text{member_no}$
 - $\text{member_no} \rightarrow id$
 - 2 Keys: $id \mid \text{member_no}$
- In BCNF
- Issues:
 - member_no is needed for issue / return queries. It is unnecessary to have faculty details with that.
 - member_no may also come from **students** relation.
 - member_type is needed for issue / return queries. This is implicit by the fact that we are in faculty relation.

Database Management Systems | Partha Pratim Das | 28/24

Member number can also come from the faculty. So, similarly member number is common with again in the faculty, it is very easy to see the design it is already in BCNF, but member number can also come from the students how do you differentiate between them.

(Refer Slide Time: 26:09)

LIS Schema Refinement (7): Query

- Consider a query:
 - Get the name of the member who has issued the book having accession number = 162715
 - ▷ If the member is a student,
 - `SELECT student_fname as First_Name, student_lname as Last_Name`
 - `FROM students, book_issue`
 - `WHERE accession_no = 162715 AND book_issue.member_no = students.member_no;`
 - ▷ If the member is a faculty,
 - `SELECT faculty_fname as First_Name, faculty_lname as Last_Name`
 - `FROM faculty, book_issue`
 - `WHERE accession_no = 162715 AND book_issue.member_no = faculty.member_no;`
 - **Which query to fire!**

Database Management Systems | Partha Pratim Das

So, that is where you get into the problem with writing a query. Suppose, you have to get the name of the member who is issued the book having accession number such and such. Now, where would you have to get it from? So, you have to get it from the book issue. Now, if you have to get it from the book issue, So, you are saying accession number is equal to this, this and now, the question is what do you say where do you get the, with this will give you the membership number from the book issue you will get the membership number, but how do you get these details?

Because from the membership number you do not know, whether the member is a student or is a faculty. If he is a student, then you will have to look in the students table, if it is a faculty you have to look in the faculty table. So, you can see that, we created a design where everything has been made into BCNF, no redundancy, nothing, but we have missed out somewhere so that we cannot effectively write the simplest query that we need to write; who has issued a book? You do not know which query to fire.

(Refer Slide Time: 27:30)

The screenshot shows a presentation slide titled "LIS Schema Refinement (8): members". The slide content is as follows:

There are 4 categories of members: ug students, grad students, research scholars, and faculty members. This leads to the following specialization relationships:

- Consider the entity set **members** of a library and refine:
 - Attributes:
 - member_no ✓
 - member_class – 'student' or 'faculty', used to choose table ✓
 - member_type – ug, pg, rs, fc, ...
 - roll_no (if member_class – 'student'. Else null)
 - id (if member_class – 'faculty'. Else null)
- We can then exploit some hidden relationship:
 - students IS_A members
 - faculty IS_A members
- Type of relationship
 - One-to-one

In the bottom right corner of the slide, there is a small video feed of a man, presumably the lecturer, speaking.

So, we need refinement. So, how can we refine? We have to see that the members what we have missed out is members need to know more, members need to know whether a member is a student or a faculty this information is nowhere there. So, I have to basically search both these tables and later on if I have staff members a third table to see given a particular membership number whether that person is a student or a faculty or a staff.

So, what would be better certainly is to expand the member relationship. So, along with the member number you have something this is now your design attributes are coming this was not stated. Now, you are saying whether member class is student or it is faculty, which we will use to choose the table for the answering the queries.

Then member type will be there and to relate to the details of whether it is student or a faculty you put the roll number or the ID number naturally both cannot be there. I mean rarely, rarely a faculty will also be a student, but usually both will not be there. So, it should be nullable fields. And here by designing this, why did you get this from, it is nowhere written but we got it from the, again reading between the lines that in the context of the library a student is a member. So, and a faculty is a member too. So, this is a generalization available, which makes this type of relationship possible and naturally this is one to one.

(Refer Slide Time: 29:20)

The screenshot shows a presentation slide from IIT Madras. The title is "LIS Schema Refinement (9): Query". The content includes a bullet point: "Consider the access query again:" followed by a sub-bullet: "Get the name of the member who has issued the book having accession number = 162715". Below this is a SQL query:

```
SELECT
((SELECT faculty_fname as First_Name, faculty_lname as Last_Name
FROM faculty
WHERE member_class = 'faculty' AND members.id = faculty.id)
UNION
(SELECT student_fname as First_Name, student_lname as Last_Name
FROM students
WHERE member_class = 'student' AND members.roll_no = students.roll_no))
FROM members, book_issue
WHERE accession_no = 162715 AND book_issue.member_no = members.member_no;
```

 The query is annotated with blue lines and curly braces. Braces group the faculty and student sub-queries, and the final FROM clause. Blue lines underline the 'book_issue' table and the 'accession_no' and 'member_no' fields in the WHERE clause.

So, once you have made that change, now it gets easier to write the query because you see if member class is faculty or so you have the accession number, you have the member number available from that. So, you check whether this is in student or this is in faculty, any one of them will be not null, take a union and you get the final result.

(Refer Slide Time: 29:54)

IT Madras
LIS Schema Refinement (10): members

Module 28
Partha Pratim Das

- **members**(member_no, member_class, member_type, roll_no, id)
 - member_no → member_type, member_class, roll_no, id
 - member_type → member_class
 - Key: member_no

Database Management Systems Partha Pratim Das

So, this was just a quick way to show that well, you can have, you may need to extend the design by for supporting the right queries and for doing the right design. So, this is your new members design.

(Refer Slide Time: 30:11)

IT Madras
LIS Schema Refinement (11): students

Module 28
Partha Pratim Das

- **students**(student_fname, student_lname, roll_no, department, gender, mobile_no, dob, degree)
 - roll_no → student_fname, student_lname, department, gender, mobile_no, dob, degree
 - Keys: roll_no
 - Note:
 - ▷ member_no is no longer used
 - ▷ member_type and member_class are set in **members** from degree at the time of creation of a new record.

Database Management Systems Partha Pratim Das

Similarly, your students design does not need member number anymore, because you have the roll number and member entity is taking care of that.

(Refer Slide Time: 30:23)

The slide is titled "LIS Schema Refinement (12): faculty". It lists the attributes for the **faculty** table: `faculty_fname`, `faculty_lname`, `id`, `department`, `gender`, `mobile_no`, and `doj`. It specifies that `id` is the primary key, derived from `faculty_fname`, `faculty_lname`, `department`, `gender`, `mobile_no`, and `doj`. A note states that `member_no` is no longer used, and `member_type` and `member_class` are set in the **members** table at the time of creating a new record. The slide is part of a presentation by Partha Pratim Das, with a sidebar showing a navigation menu and a video feed of the presenter at the bottom right.

- **faculty**(`faculty_fname`, `faculty_lname`, `id`, `department`, `gender`, `mobile_no`, `doj`)
 - `id` → `faculty_fname`, `faculty_lname`, `department`, `gender`, `mobile_no`, `doj`
 - Keys: `id`
 - Note:
 - ▷ `member_no` is no longer used
 - ▷ `member_type` and `member_class` are set in **members** at the time of creation of a new record

Faculty also does not need the member number. So, they get simplified.

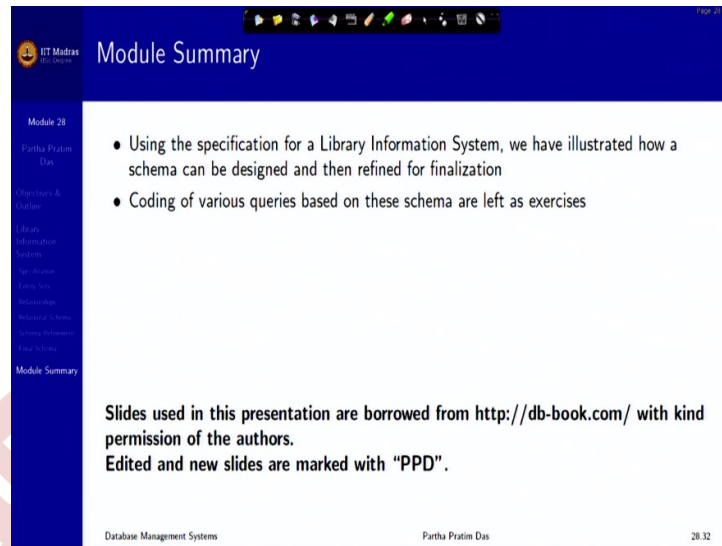
(Refer Slide Time: 30:28)

The slide is titled "LIS Schema Refinement (13): Final". It lists the final tables in the schema: **book_catalogue**, **book_copies**, **book_issue**, **quota**, **members**, **students**, **faculty**, and **staff**, each with their respective attributes. The slide is part of a presentation by Partha Pratim Das, with a sidebar showing a navigation menu and a video feed of the presenter at the bottom right.

- **book_catalogue**(`title`, `author_fname`, `author_lname`, `publisher`, `year`, `ISBN_no`)
- **book_copies**(`ISBN_no`, `accession_no`)
- **book_issue**(`member_no`, `accession_no`, `doi`)
- **quota**(`member_type`, `max_books`, `max_duration`)
- **members**(`member_no`, `member_class`, `member_type`, `roll_no`, `id`)
- **students**(`student_fname`, `student_lname`, `roll_no`, `department`, `gender`, `mobile_no`, `dob`, `degree`)
- **faculty**(`faculty_fname`, `faculty_lname`, `id`, `department`, `gender`, `mobile_no`, `doj`)
- **staff**(`staff_fname`, `staff_lname`, `id`, `gender`, `mobile_no`, `doj`)

And you have finally, this relational schema that there could be more and more refinements you can do, but this is a sufficiently decent schema everything is in BCNF no redundancy in that respect. And at least, one type of query you have tried out to sort out this concept of member coming in which was not explicitly mentioned to be having so many different attributes.

(Refer Slide Time: 30:57)



Module Summary

- Using the specification for a Library Information System, we have illustrated how a schema can be designed and then refined for finalization
- Coding of various queries based on these schema are left as exercises

Slides used in this presentation are borrowed from <http://db-book.com/> with kind permission of the authors.
Edited and new slides are marked with "PPD".

Database Management Systems Partha Pratim Das 28/32

So, this has been an example so, you please work on this, try to understand, make alternate designs and try to code the query so that you can get a better confidence on this design process. Thank you very much for your attention and we meet in the next module to discuss further on the design of relational databases.

